Multi-Expert Agentic Pipeline for Peptide Generation

Fitger Nathan, Iliescu Eduard, Macdonald Arthur, Roudeau Niels
Université de Bordeaux, Master 2, 2026
Supervisors: Akka Zemmari, Enzo Eddebbarh, Johan Samot

2026

Abstract

We present a proof-of-concept multi-expert agentic pipeline for *in silico* peptide generation [7]. A central orchestrator coordinates three specialized agents: an AI Designer, a Chemistry Constraints agent, and a Biology agent providing an embedding-based ranking signal [8]. The generator is based on variational architectures (VAE/CVAE), and chemistry and biology scores are fused by the orchestrator to build a global Top-K ranking across iterations.

Keywords: Multi-Agent Systems, Peptide Generation, In Silico Drug Design, VAE/CVAE, Evolutionary Scale Modeling.

1 Introduction

Peptides, and especially antimicrobial peptides (AMPs), are promising candidates for therapeutic, agricultural, and industrial applications [7]. Their *in silico* design is naturally framed as an optimization loop: generate candidate sequences, apply feasibility constraints, and rank candidates with proxy signals for expected function and risk [7, 8]. The central difficulty is multi-objective control: improving one criterion (for example activity proxy) can degrade others (such as developability).

This report presents a multi-expert agentic proof of concept that addresses this difficulty through explicit role decomposition. The pipeline combines three specialized agents and one orchestrator: a generator that proposes candidates, a chemistry module that evaluates physicochemical admissibility, and a biology module that provides an embedding-based ranking signal. The orchestrator enforces an iterative cycle (propose → filter → score → revise), aggregates expert feedback, and maintains traceable run logs.

Our objective is an AI-systems evaluation rather than biological validation. We therefore focus on reproducible proxy-based metrics: validity, Top-K quality, diversity, constraint pass rate, and score evolution across rounds, followed by comparison with a non-agentic baseline. The guiding question is whether iterative multi-expert coordination yields measurable gains over one-shot conditional generation.

The remainder of the report is organized as follows. Section 2 positions the work in the recent literature and project scope. Section 3 details the pipeline design and agent responsibilities. Section 4 describes scoring, metrics, and baseline protocol. Section 6 reports empirical behavior and baseline comparison. Section 7 discusses limits and extensions, and Section 8 closes the report.

2 Context and Related Work

2.1 Evolution of In Silico Peptide Generation

Computational peptide design has moved from rule-based filtering toward data-driven generation and ranking pipelines [1]. Earlier approaches relied primarily on handcrafted physicochemical descriptors and QSAR-style models; modern methods combine these descriptors with learned sequence representations. In practice, descriptor-based rules remain useful because they encode explicit developability constraints, while learned models improve exploration of large sequence spaces.

This evolution improved throughput and candidate quality, but did not remove the core optimization bottleneck: peptide design is intrinsically multi-objective. Activity, safety, synthesizability, and stability are only partially aligned, and a single monolithic model often fails to satisfy all objectives simultaneously.

2.2 The Shift to Agentic Workflows

Despite strong progress, many pipelines still treat generation, chemistry filtering, and biology scoring as loosely connected stages [6]. This fragmentation limits feedback: one-shot generation may satisfy one criterion while failing others, with no structured mechanism to revise candidates.

Agentic workflows address this issue by organizing optimization as a closed loop: propose, evaluate, revise, and repeat [3]. The key contribution is not a new single predictor but a coordination policy across complementary experts. For constrained peptide design, this makes trade-offs explicit at each round and supports controlled exploration–exploitation behavior [1].

2.3 Project Scope and Alignment

The present project follows this direction as an AI-systems proof of concept. The deliverable is an end-to-end pipeline with three expert agents (generation, chemistry, biology) coordinated by an orchestrator

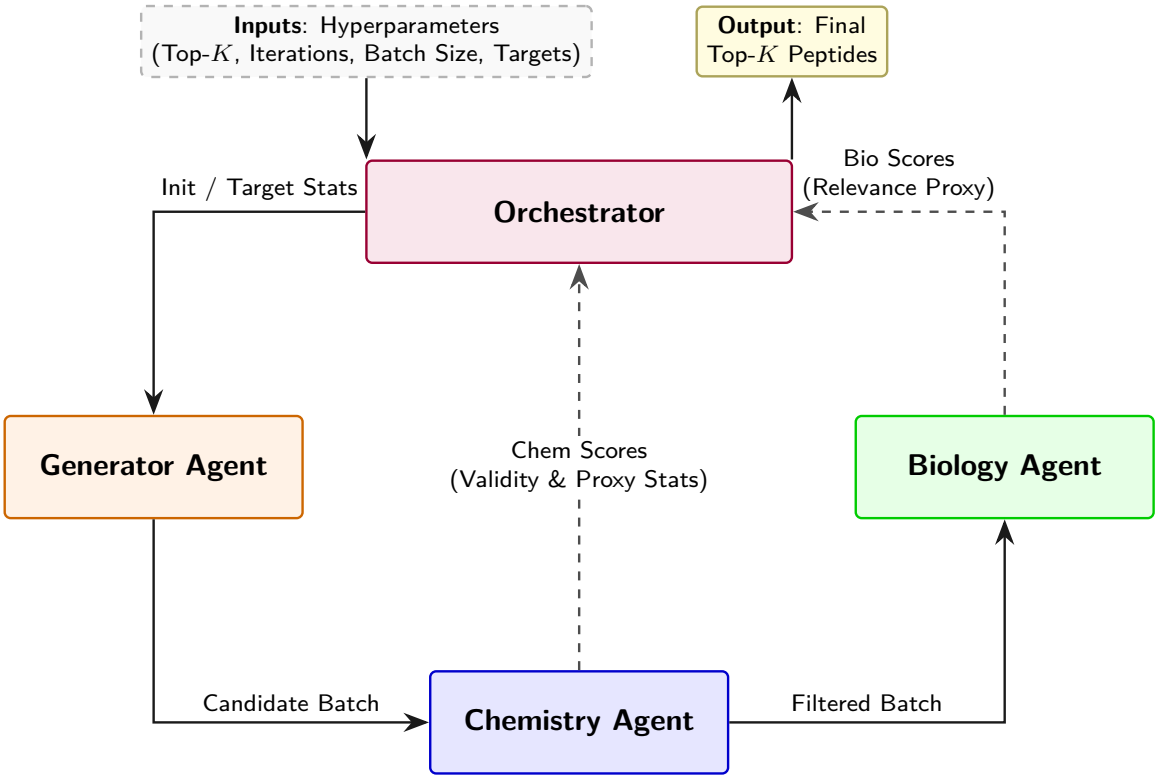

Figure 1: Agentic Pipeline Architecture. Solid lines denote the sequential flow of peptide candidate batches. Dashed lines indicate the feedback loops, where both the Chemistry and Biology agents return their respective proxy scores back to the Orchestrator. Color coding differentiates the specialized domains of the pipeline.

over multiple rounds. The goal is to test whether explicit role decomposition and iterative coordination improve optimization behavior under practical proxy constraints.

The scope is strictly *in silico*. Scores are treated as documented proxies, not experimental evidence, and performance is evaluated through reproducible quantitative metrics and comparison against a non-agentic baseline. In this setting, success is defined as measurable improvement from coordination and iteration.

3 Multi-Agent System Architecture

The global pipeline is organized around three specialized agents coordinated by a central orchestrator. Figure 1 summarizes the end-to-end data flow and feedback loops in the iterative process.

3.1 The Orchestrator

The orchestrator is the control layer that turns the three experts into one iterative optimization system. At each round it:

1. Generate a candidate batch from target constraints (user-provided or inferred from chemistry configuration).

2. Evaluate candidates with the Chemistry Agent (descriptor extraction, score, in-limits status). If no in-limits peptide is found, the round still proceeds with the available candidates to avoid loop collapse.

3. Score candidates with the Biology Agent, fuse expert signals, and update a global best-by-sequence ranking used both for parent selection and final Top-K output.

This combines exploration (new proposals) and exploitation (reuse of high-ranked candidates) under one policy and a fixed iteration budget.

Two implementation choices are central to robustness and comparability:

- **Global ranking across rounds:** one best entry is kept per unique sequence, so strong early candidates remain competitive.
- **Round continuity under strict filtering:** biology scoring can still be applied when chemistry yields no in-limits peptides, preventing starvation.

The orchestrator also records round-wise events for traceability:

- Input settings
- Filtered populations
- Expert scores
- Selected candidates

This log-centric structure makes optimization paths auditable end to end.

3.2 Generator Agent

The final generator used in the pipeline is a Conditional Variational Autoencoder (CVAE). An unconditional VAE was tested first, but because it has no target-conditioning mechanism, sampled sequences were too random with respect to the requested profile and did not provide useful candidates for iterative optimization.

The CVAE operates on fixed-length peptide sequences encoded as one-hot vectors over the 20 canonical amino acids. For a peptide of length L, the sequence input dimensionality is $20 \times L$. Unlike unconditional generation, the CVAE conditions both encoder and decoder on an auxiliary property vector y, and therefore learns:

$$q_\phi(z \mid x, y), \qquad p_\theta(x \mid z, y). \tag{1}$$

In the implemented model, the condition vector is concatenated with the peptide input before encoding and concatenated again with the latent representation before decoding. This design allows generation to remain consistent with the training distribution while being biased toward requested physicochemical targets. The condition vector includes sequence-level and physicochemical descriptors (for example length, molecular weight, charge-related terms, hydrophobicity-related terms, and developability proxies such as $\log P$, hydrogen-bond counts, and TPSA).

In the current implementation, each property is encoded as a scalar value. Missing entries are replaced by predefined defaults, user constraints override these defaults, and the final vector is padded or truncated to match the configured conditioning dimensionality. This makes generation both plausible and directed toward downstream chemistry filtering.

Physicochemical Conditioning

The CVAE conditioning space includes 12 descriptors, while the current Chemistry Agent actively evaluates six configurable properties (length, molecular weight, net charge, isoelectric point, hydrophobicity, and logP). This means that generation can be guided by a broader condition vector than the one strictly validated by chemistry filtering.

This link between chemistry-oriented descriptors and the generative model provides a practical way to translate biochemical objectives into numerical conditioning variables. As a result, the CVAE can be integrated into a broader peptide design pipeline in which candidate peptides are generated, analyzed, and filtered according to downstream criteria.

Limitations

The CVAE design also introduces practical constraints (data dependence, conditioning limits, and biological scope), but these points concern interpretation and future extensions more than architecture itself. We therefore summarize them in the discussion section (Section 7).

3.3 Chemistry Agent

The Chemistry Agent is the deterministic constraint-and-scoring stage of the pipeline. It receives peptide sequences from the generator, evaluates them against configurable physicochemical targets, and returns both feasibility status and a quantitative chemistry score.

Sequence validation and property extraction:

The agent first validates peptide strings against the 20 standard amino acids. For each valid candidate, it computes configured physicochemical properties using peptide-level functions and RDKit-based logP estimation.

Multidimensional property profiling:

To characterize developability, the agent computes a configurable set of descriptors from two complementary sources: peptide-centric properties (length, net charge at chosen pH, isoelectric point, hydrophobicity) and molecule-centric properties (notably RDKit-based $\log P$). This profile lets the orchestrator compare candidates in a unified physicochemical space.

Quantitative scoring and filtering:

The agent assigns each candidate a continuous score based on distance to target values, with optional per-property weighting. Chemistry evaluation first computes per-peptide properties and in-limit status, then returns scored candidates to the orchestrator. At orchestration level, ranking prioritizes in-limit candidates; if none are available in a round, out-of-limit candidates are still forwarded for biological scoring to preserve loop continuity. This design prevents early pipeline collapse while maintaining selection pressure toward target profiles.

The chemistry output is used directly by the orchestrator for admissibility-aware ranking and for score fusion with the biology signal.

3.4 Biology Agent (Function & Risk)

The Biology Agent provides the biological ranking signal used by the iterative loop. In this implementation, it relies on ESM-2 embeddings [2, 9] and can be conditioned by context (for example a reference peptide).

At each iteration, the agent scores the selected batch and returns values used by the orchestrator for candidate ranking and parent selection. The score is treated as a relative proxy for optimization guidance; the exact formulation is detailed in Section 4.1.

4 Experimental Setup

This section documents the implemented scoring and comparison protocol used in the current proof of concept.

4.1 Quantitative Scoring System

Chemist Agent Scoring

The chemistry workflow operates in two steps:

1. **Hard constraint check and scoring:** Candidates are evaluated against strict limits and scored from target-distance-based property scores.
2. **Fallback at orchestration level:** If no candidate satisfies all limits, out-of-limits candidates are still passed to biology scoring to keep the iterative loop active.

This policy keeps the loop active while preserving selection pressure toward developable candidates.

- **Solubility & Stability proxies:** Isoelectric point and net charge are computed for each peptide.
- **Permeability proxy:** The profile includes $\log P$ (RDKit-based) and hydrophobicity.
- **Basic developability descriptors:** Length and molecular weight are included in scoring and admissibility checks.

Biologist Agent Scoring

The objective is to rank a batch of peptides and identify which candidate is closest to the provided context (an existing peptide or requested properties represented as a reference embedding).

The biologist module uses ESM-2 embeddings (default: `facebook/esm2_t12_35M_UR50D`) with attention-mask-aware mean pooling to produce one vector per peptide.

Two proxy variants are implemented. The first uses cosine similarity, mapped to $[0, 1]$. The second (Global L2) computes Euclidean distance to a reference embedding and applies a batch-independent exponential mapping. In preliminary tests, cosine scores showed limited contrast for short peptides, so Global L2 was retained for the main experiments.

The resulting score is used by the orchestrator for candidate ranking and Top-K selection. It is treated as a proxy signal to guide search, not as an absolute biological activity measurement; biological validity would require peptide-level experimental assays.

Orchestrator-Level Scoring and Ranking

At orchestration level, each candidate receives a combined proxy score obtained by averaging Chemistry and Biology signals with equal weight:

$$S_{\text{combined}} = \frac{S_{\text{chem}} + S_{\text{bio}}}{2}. \tag{2}$$

The ranking policy then prioritizes chemically admissible peptides (in-limits) and, within that constraint status, orders candidates by combined score (with expert scores used as tie-breakers). The orchestrator maintains this ranking globally across rounds, retaining one best entry per unique sequence and returning the final Top-K from this global pool.

In practical terms, this means candidate quality is not decided only round by round: a strong peptide discovered early can remain competitive at the end if later rounds do not surpass it.

4.2 Evaluation Metrics

We report six core metrics, computed at run level and on final Top-K outputs:

- **Validity rate (%):** fraction of returned peptides that satisfy all configured chemistry constraints (`in_limits=True`).
- **Top-1 score:** best combined score among returned candidates.
- **Top-K mean score:** average combined score across final Top-K peptides.
- **Constraint pass rate:** proportion of generated peptides passing chemistry constraints before final ranking.
- **Diversity (sequence-level):** mean pairwise distance (normalized edit distance) and unique-ratio in Top-K.
- **Diversity (distribution-level):** bigram entropy over amino-acid pairs as a complementary compositional diversity proxy.

Final ranking is built globally across rounds (best-by-sequence retention), then truncated to Top-K.

4.3 Baselines

The baseline is a simpler non-agentic reference used to test whether iterative multi-expert coordination provides measurable gains. We use a standalone Conditional Variational Autoencoder (CVAE) that generates peptide candidates from conditioning features, without chemistry/bio critique loops or orchestration rounds. Training data are fetched from DBAASP and retain sequence-level and physicochemical conditioning fields (length, pH, molecular weight, logP, net charge, isoelectric point, hydrophobicity, cationicity).

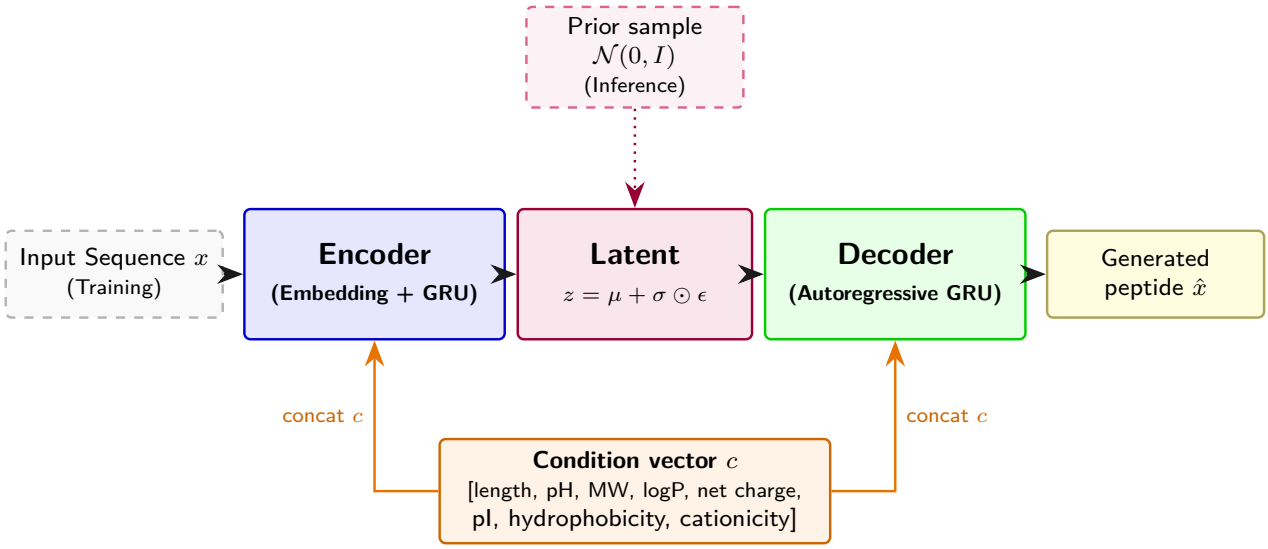

Figure 2: Baseline CVAE architecture used for comparison. During training, the model learns $q_\phi(z \mid x, c)$ and reconstructs sequences with an autoregressive decoder conditioned on z and c. During inference, peptides are generated by sampling $z \sim \mathcal{N}(0, I)$ and decoding once, without iterative multi-agent feedback.

Many API entries contain missing values, so absent fields are replaced with fixed defaults before standardization; this is imperfect but provides a stable comparison anchor. The baseline is a GRU-based CVAE: the encoder maps sequence-condition pairs to latent Gaussian parameters, and the autoregressive decoder reconstructs sequences conditioned on latent code and property vector. Training uses cross-entropy reconstruction, KL regularization, Adam, and annealed β. At inference, candidates are sampled from the latent prior under a scaled target condition vector, with temperature and top-k sampling controlling exploration. Unlike the full pipeline, this baseline performs one-shot conditional generation without round-level feedback.

5 User Interface (Streamlit)

The project uses a Streamlit application (`streamlit_app/app.py`) as the main user-facing interface to the multi-expert peptide pipeline. Its primary purpose is to make experimentation reproducible and accessible by exposing model selection, parameterization, execution, and result inspection in one place.

5.1 Purpose of the UI

The Streamlit UI acts as an orchestration console for research workflows. Instead of running separate scripts for each stage, users can configure and launch end-to-end tasks from a single screen. This reduces operational friction, standardizes run configuration, and supports faster iteration during model development and comparison.

5.2 Core Functionality

The interface is organized around five functional views: *Orchestrator*, *Train Models*, *Generator Tests*, *Chemist Tests*, and *Biologist Tests*. A sidebar menu provides navigation between these modules and includes shared runtime controls (notably the global log level), which are persisted in `st.session_state`.

UI controls are generated from structured configuration (`streamlit_app/config.json`) and from schema-driven definitions (e.g., `ChemistConfig`), allowing users to adjust generator, biologist, and chemistry constraints without modifying source code. The chemistry forms support both scalar entries and range/target constraints, which aligns the interface with constrained peptide design scenarios.

To improve transparency during execution, the app includes a real-time log panel through a custom `StreamlitLogHandler`. This gives immediate feedback on pipeline behavior, while validation-oriented visual cues help identify invalid or incomplete inputs.

Overall, the Streamlit layer provides the practical bridge between pipeline components and day-to-day experimentation needs, with emphasis on usability, traceability, and rapid iterative testing.

Representative screenshots of the Streamlit application are provided in the annex: Pipeline interface run panel (Figure 6), biologist testing view (Figure 7), training and generation panel (Figure 8), logging interface (Figure 9), and export interface (Figure 10), menu and navigation (Figure 11).

6 Results and Analysis

This section summarizes the main quantitative findings from pipeline-only studies and from comparison with the non-agentic baseline.

6.1 Pipeline Sensitivity Under Fixed Resources

The two panels in Figure 3 show the same structural trade-off: larger search budgets improve quality, but runtime grows much faster than score.

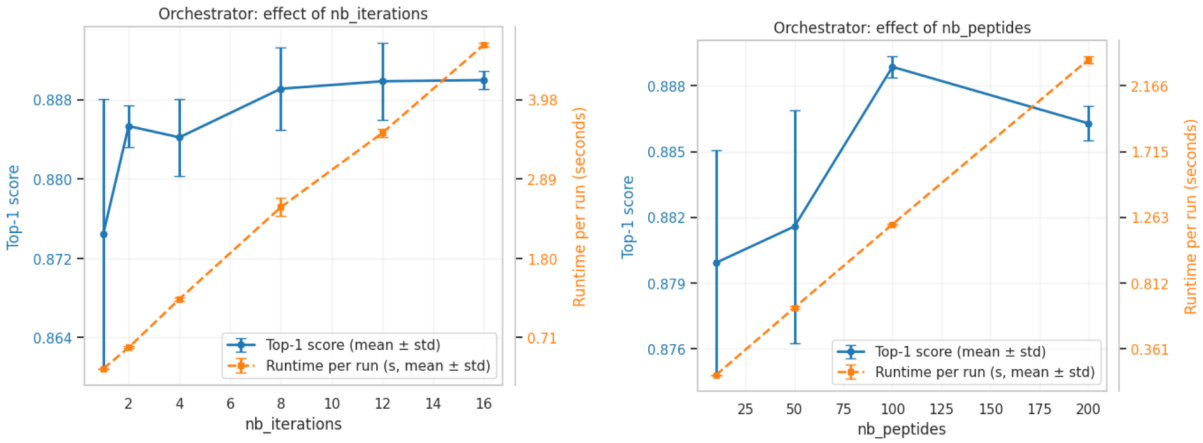

Figure 3: Pipeline sensitivity to orchestrator budget: effect of the number of iterations (left) and the number of generated peptides per iteration (right) on Top-1 score and runtime.

When the number of iterations increases from 5 to 75, the Top-1 score moves only from 0.88 to 0.89, while runtime rises from 0.92 s to 13.39 s (about 14.60×). This very slow quality growth strongly suggests that the current orchestrator policy is not extracting enough additional value from later rounds. In other words, extra rounds are expensive, but they are not yet sufficiently informative.

The same interpretation appears when increasing the number of generated peptides per round from 64 to 512: Top-1 score moves from 0.88 to 0.89, while runtime rises from 1.83 s to 14.70 s (about 8.00×). The search is stable and does improve quality, but the gain per unit of compute remains small at high budgets.

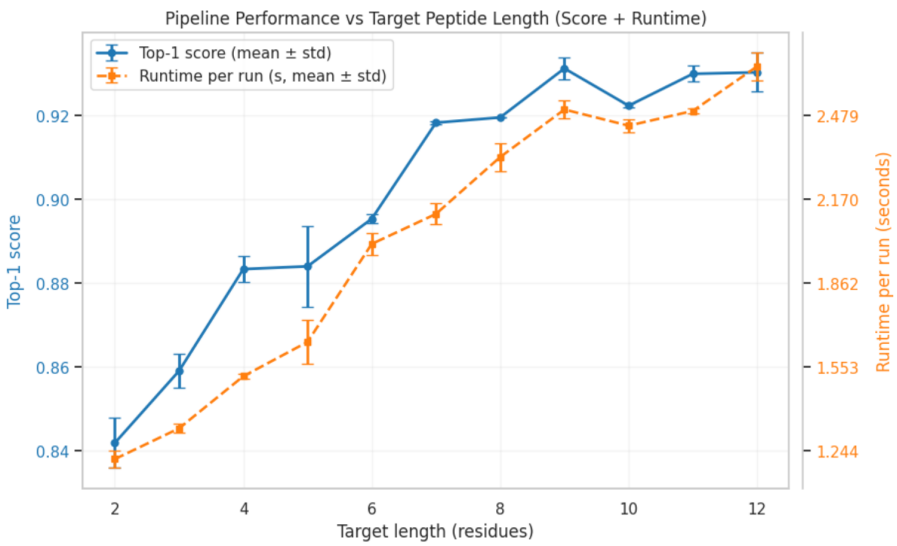

Figure 4: Pipeline-only stress test: effect of target length on Top-1 score and runtime.

We know from the chemist information that peptides with small length are harder to find and very rare. Figure 4 confirms that target length controls difficulty. Scores are lower at extreme short lengths and then rise toward a plateau for longer peptides: around 0.82 at lengths 2–3, around 0.93 near length

11. Runtime grows more smoothly, from about 1.90 s (length 2) to 3.77 s (length 12).

This result should be interpreted carefully: obtaining around 0.82 even for length-2 targets can be seen as robustness, but it can also indicate that the biological ranking signal is not sufficiently discriminative in pathological short-length regimes. Said differently, part of this score may reflect scorer tolerance rather than truly meaningful biological plausibility, which is consistent with the proxy nature of the current evaluator.

6.2 Pipeline vs Baseline by Target Length

The comparison evaluates four methods for target lengths 2 to 11: non-agentic baseline, pipeline with 1 iteration, pipeline with 3 iterations and pipeline with 10 iterations.

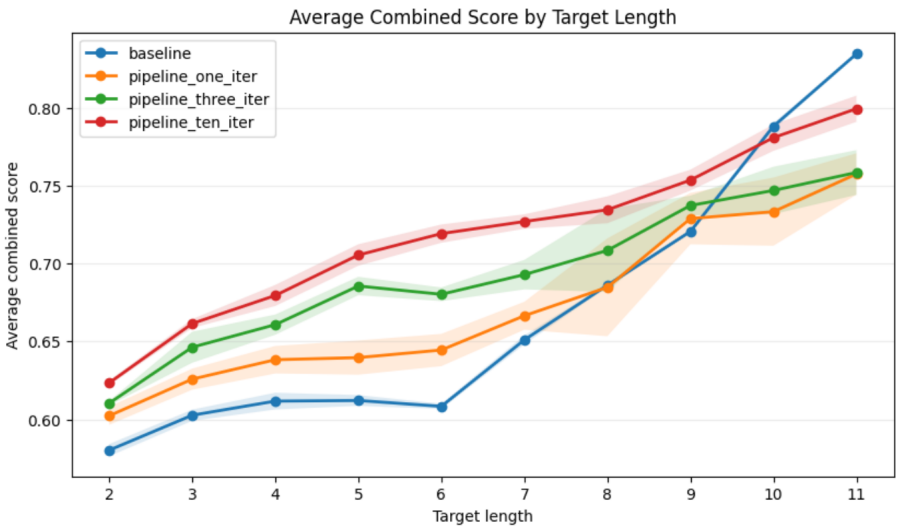

Figure 5: Average combined score by target length (baseline vs iterative pipeline variants).

The score trend in Figure 5 is clearly regime-dependent.

For hard short targets, iteration gives a clear advantage. At length 2, the baseline is at 0.58, while the ten-iteration pipeline reaches 0.62. At length 3, the baseline is 0.60 and the ten-iteration pipeline is 0.66. The same pattern persists through mid lengths; for example at length 6, the baseline is 0.61 and the ten-iteration pipeline is 0.72.

For easier long targets, the picture changes. At length 11, the baseline reaches 0.83, above the ten-iteration pipeline at 0.80. This means iterative critique is not universally superior; it is most useful in difficult regimes, and can be unnecessary or slightly detrimental when direct generation is already strong.

Important clarification: Figure 4 reports Top-1 score, which measures the best candidate found in a run while Figure 5 reports an average combined score, which reflects the global quality of many generated candidates for the same target length.

This is why values can differ substantially for the same target length. A method can obtain a strong Top-1 by finding a few very good candidates, while still producing many mediocre candidates that lower the average. In the current experiments, this gap suggests that search and ranking can still identify good outliers, but population-level quality remains uneven, especially in short-length regimes.

6.3 Interpretation

Overall, the most robust conclusion is that iterative multi-expert search is most valuable in difficult target regimes, where it improves the score relatively to the non-agentic generation. The same experiments also expose clear improvement directions: a more sample-efficient orchestration strategy (to reduce slow score growth with additional rounds) and a more discriminative biological scorer for extreme short-length cases.

7 Discussion & Future Work

7.1 Domain Feedback and Scope Clarification

Overall, expert feedback is consistent with the quantitative results and helps clarify the scope of our claims. Chemistry-side outputs are considered feasible within the configured design space, whereas biological efficacy cannot be established without wet-lab assays. Accordingly, the current system should be interpreted as a robust *in silico* optimization proof of concept.

7.2 Current Generator Limitations

At generator level, two limitations remain important for interpretation.

First, generation assumes fixed-length peptide inputs because one-hot encoding and decoder output size depend directly on sequence length. Supporting variable-length peptides would require architectural changes or additional preprocessing.

Second, conditioning quality depends on label quality. If training targets are noisy, missing, or weakly informative, the learned relation between requested properties and generated sequences becomes unstable, which reduces controllability at inference time.

7.3 Chemist Agent Improvements

Current chemistry filtering and scoring are sequence-derived. A natural extension is to incorporate richer molecular representations (for example SMILES-derived descriptors) as additional conditioning and evaluation signals. This direction would require dedicated preprocessing, careful descriptor selection, and retraining so that conditioning variables and chemistry constraints remain consistent.

7.4 LLM-Guided Input Specification

Another promising direction is to use a large language model as an interface layer for target specification. Instead of manually setting every constraint, users could express design objectives in natural language and map them to structured optimization settings. This could improve usability and make iterative design more flexible when requirements evolve.

7.5 Exploration-Oriented Orchestrator

Another key direction is to redesign orchestration to be more open to exploration, especially in difficult target regimes. Current behavior suggests that later rounds can become too conservative around already good candidates. A more exploration-oriented policy (for example adaptive exploration rate, uncertainty-aware parent selection, or diversity-driven candidate injection) could improve coverage of sequence space while preserving exploitation of strong leads.

This would directly target one of the main observed limitations: slow score gains when increasing iteration budget. The objective is not random search, but controlled exploration that improves both best-candidate quality and population-level quality.

7.6 Peptide-Specific Biological Scoring

The most impactful upgrade is on the biology side. The current ESM-based proxy is useful for ranking, but it is not biological ground truth. Replacing it with a peptide-specialized, target-conditioned predictor trained (or fine-tuned) on peptide-level validation data would directly improve ranking reliability.

Beyond a single scalar score, such a model could provide richer outputs (for example expected activity and risk-related signals), making optimization decisions more informative. Although this is a substantial research effort, it remains the clearest path toward stronger biological relevance in future versions of the pipeline.

Another issue with the biological evaluation is the differences between well known strains and unstudied ones. The currents models are trained on a limited set of bacterial species, which may not generalize well to novel or less-characterized pathogens. The models existing as of now are not great at extrapolating to new or unknown species, which is a problem for real-world applications where emerging pathogens are common.

8 Conclusion

This report presented a multi-expert agentic pipeline for *in silico* peptide design, combining generation, chemistry constraints, and biology scoring under iterative orchestration. Across experiments, the system showed stable optimization behavior: iterative rounds improved ranking quality in difficult regimes (especially short targets).

At the same time, experiments show that quality gains from larger orchestration budgets are modest relative to runtime growth, which indicates that the current orchestrator remains improvable in sample efficiency. Results also show a recurring gap between best-candidate performance (Top-1) and population-level average score.

The main contribution is architectural and methodological: explicit expert roles, global ranking across rounds, and fully traceable decision flow. This makes optimization trade-offs observable, supports reproducible comparisons, and provides a practical framework for controlled experimentation under proxy constraints.

The current biology signal remains a proxy and must not be interpreted as experimental evidence. This is particularly important in extreme short-length regimes, where score interpretation remains uncertain. Accordingly, the present work is an AI engineering proof of concept, with the most impactful next steps being peptide-specialized biological predictors, richer chemistry-aware representations, and an orchestrator that is more open to controlled exploration while still maintaining strong ranking pressure.

Code, experiment scripts, configurations, and run logs are provided in the project repository [10] to support reproducibility and future extensions.

Bibliography

[1] F. Wan, F. Wong, J. J. Collins, and C. de la Fuente-Nunez, "Machine learning for antimicrobial peptide identification and design," *Nature Reviews Bioengineering*, Feb. 2024. [Online]. Available: https://doi.org/10.1038/s44222-024-00152-x

[2] A. Rives, J. Meier, T. Sercu, S. Goyal, Z. Lin, J. Liu, D. Guo, M. Ott, C. L. Zitnick, J. Ma, and R. Fergus, "Biological Structure and Function Emerge from Scaling Unsupervised Learning to 250 Million Protein Sequences," *Proceedings of the National Academy of Sciences*, vol. 118, no. 15, p. e2016239118, 2021.

[3] Z. Xi, et al., "The Rise and Potential of Large Language Model Based Agents: A Survey," *arXiv preprint arXiv:2309.07864*, 2023.

[4] T. Guo, et al., "Large Language Model based Multi-Agent Systems: A Survey of Progress and Challenges," *arXiv preprint arXiv:2401.03428*, 2024.

[5] D. A. Boiko, R. MacKnight, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[6] K. Huang, et al., "Biomni: A General-Purpose Biomedical AI Agent," *bioRxiv*, 2025.

[7] L. Wang *et al.*, "Machine learning for peptide design," *Nature Computational Science*, vol. 2, pp. 100–112, 2022.

[8] D. Veltri, U. Kamath, and A. Shehu, "Deep learning improves antimicrobial peptide recognition," *Bioinformatics*, vol. 34, no. 16, pp. 2740–2747, 2018.

[9] Facebook Research, "ESM: Evolutionary Scale Modeling," GitHub repository. [Online]. Available: `https://github.com/facebookresearch/esm`

[10] "multi-expert-peptide-pipeline," GitHub repository, 2026. [Online]. Available: `https://github.com/Roirtur/multi-expert-peptide-pipeline`

9 Annex: Streamlit UI Screenshots

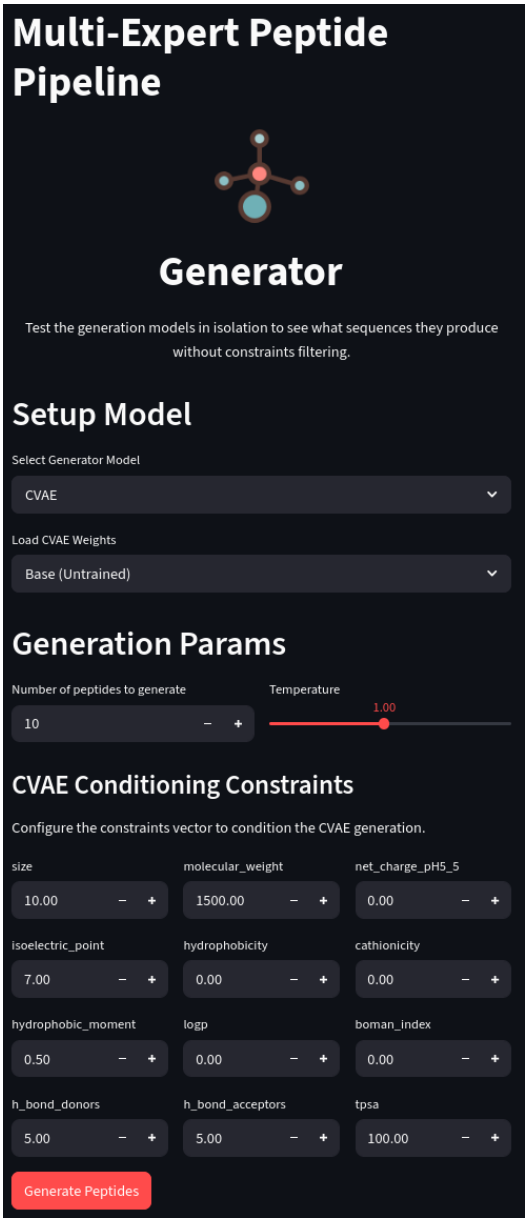

Figure 6: Generator Tests view (referenced in Section 5).

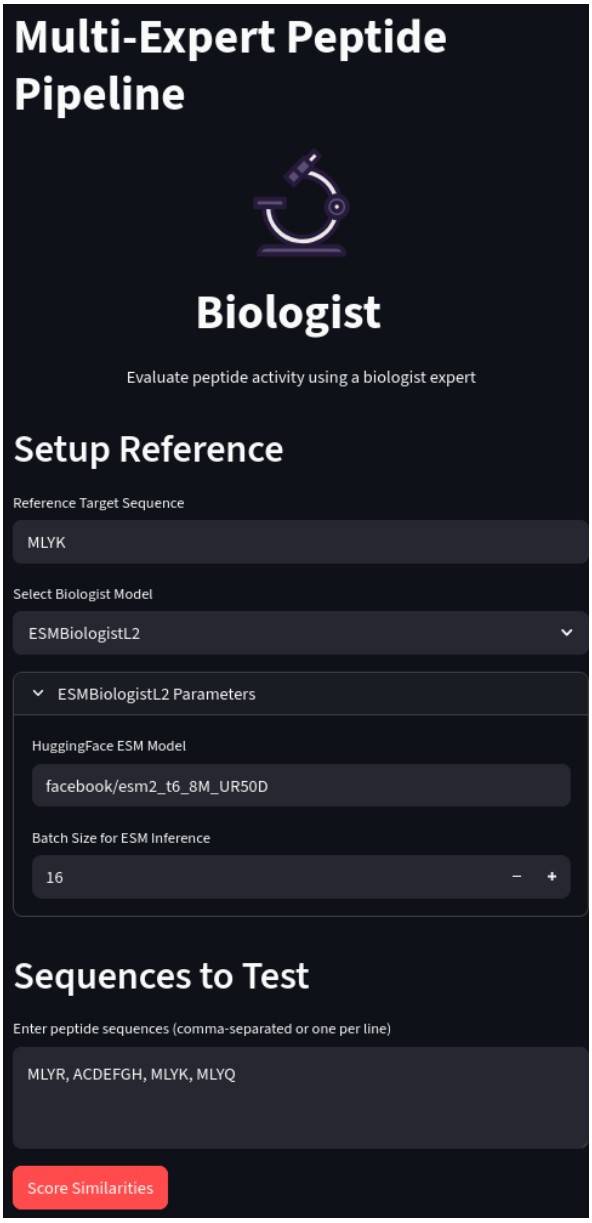

Figure 7: Biologist Tests view (referenced in Section 5).

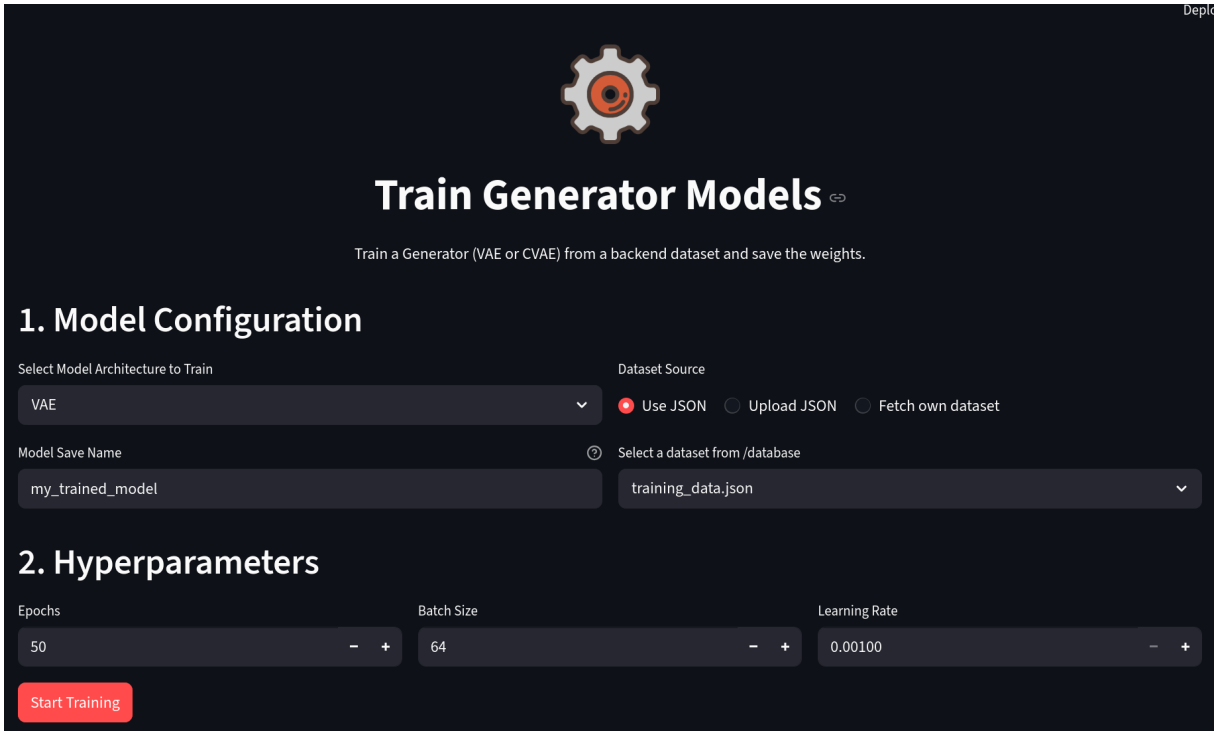

Figure 8: Train Models and generation controls (referenced in Section 5).

Figure 9: Real-time logging panel in the UI (referenced in Section 5).

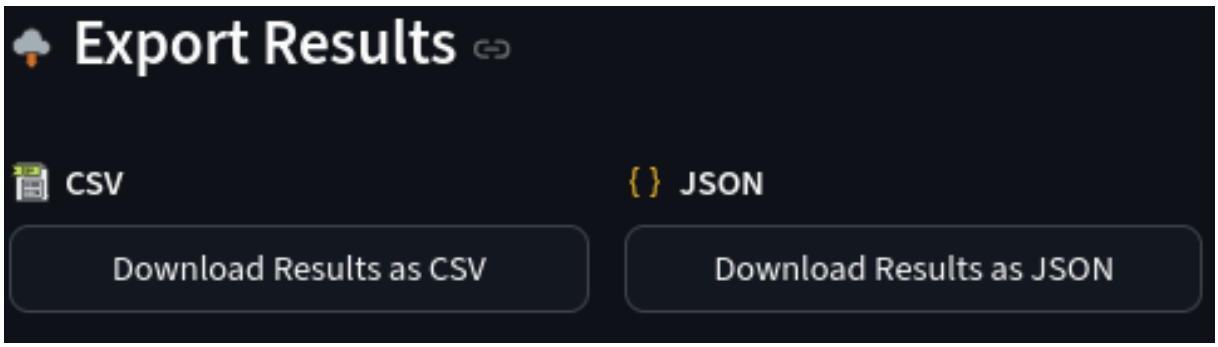

Figure 10: Export interface for experiment outputs (referenced in Section 5).

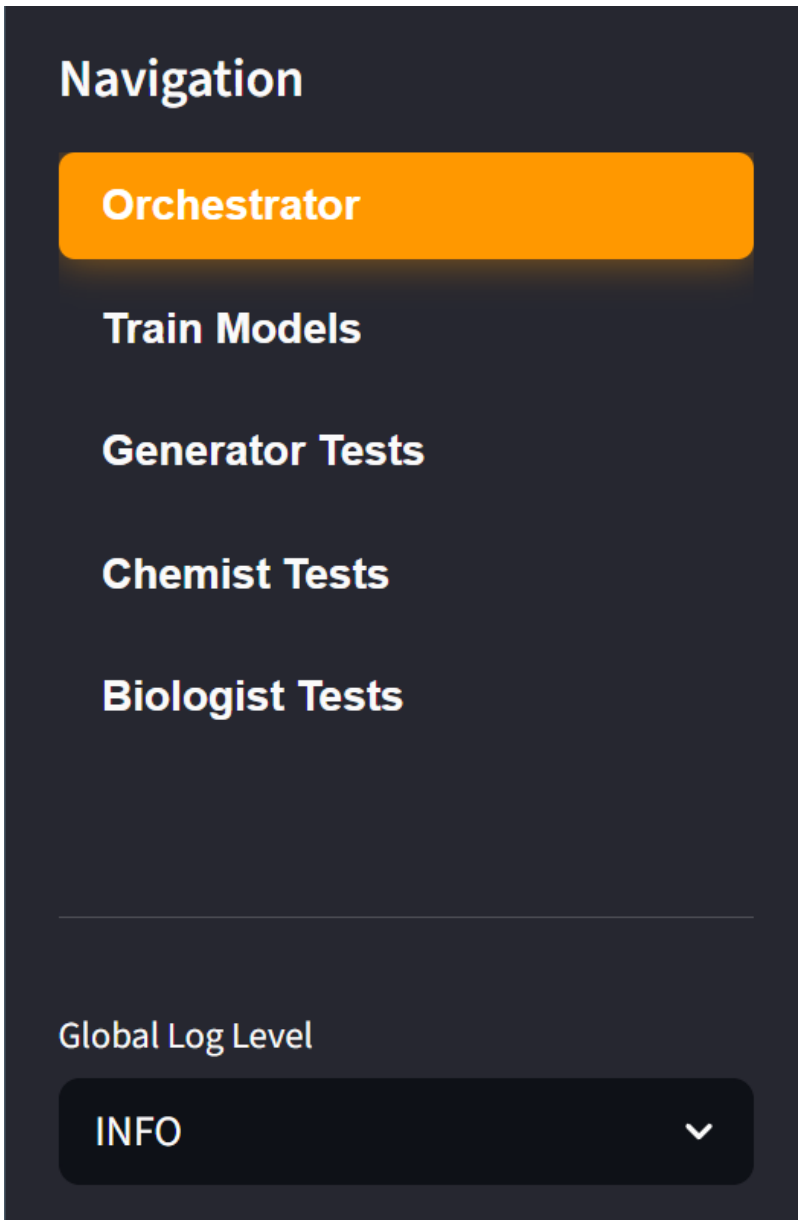

Figure 11: Streamlit sidebar menu navigation module and logging level selection (referenced in Section 5).